Introduction to Applied Machine Learning

07: Convolutional Neural Networks and Object Detection

🎓 Machine Learning Adventure 🚀

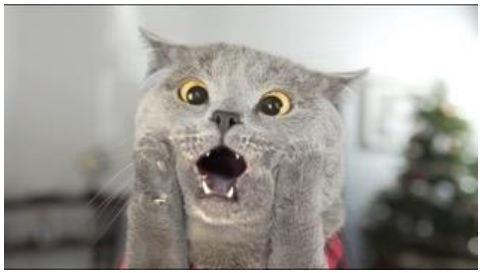
1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

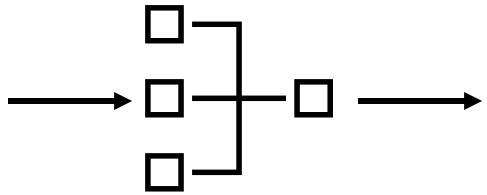
Today's Learning Goals

1. **Part 1: More on training** neural networks, including:
   - Reverse Mode Automatic Differentiation
   - PyTorch API
   - GPU Acceleration
   - Transfer learning by:
     - Feature Extraction, and
     - Fine-tuning
2. **Part 2: Object detection**, including by:
   - Sliding Windows
   - YOLO (or direct prediction)
   - Image Segmentation

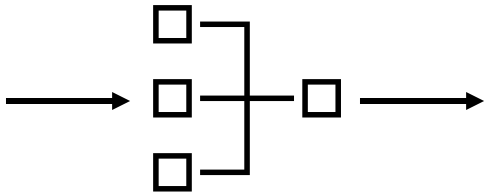
Last Time: Image Recognition (Classification)

We want a classification model (typically as a conditional probability estimator) that takes an image as input and predicts a categorical label. For examples…



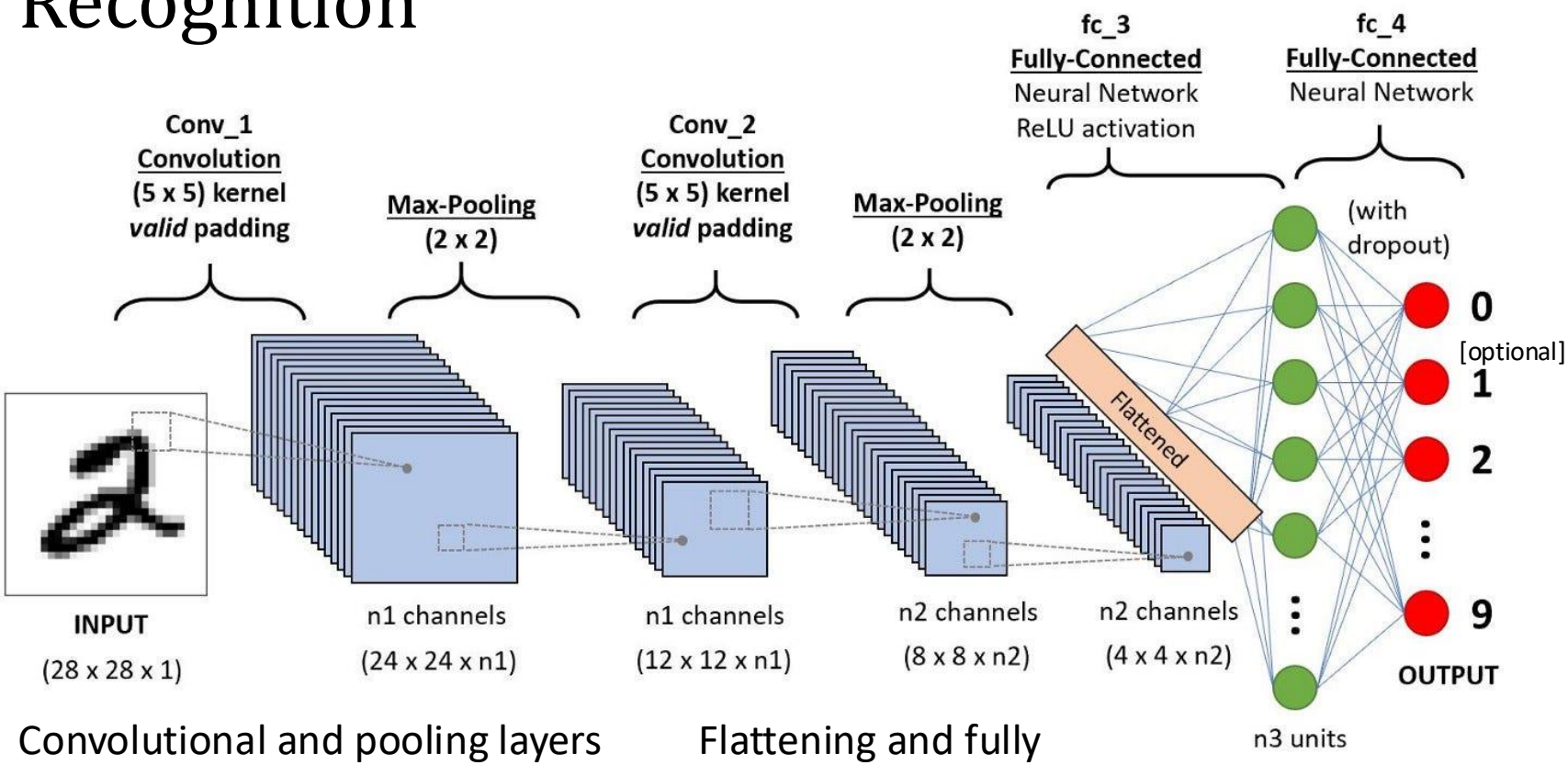


70% probability “cat”



90% probability “true” is your face

The Big Picture: Example Convolutional Neural Network Architecture for Image Recognition



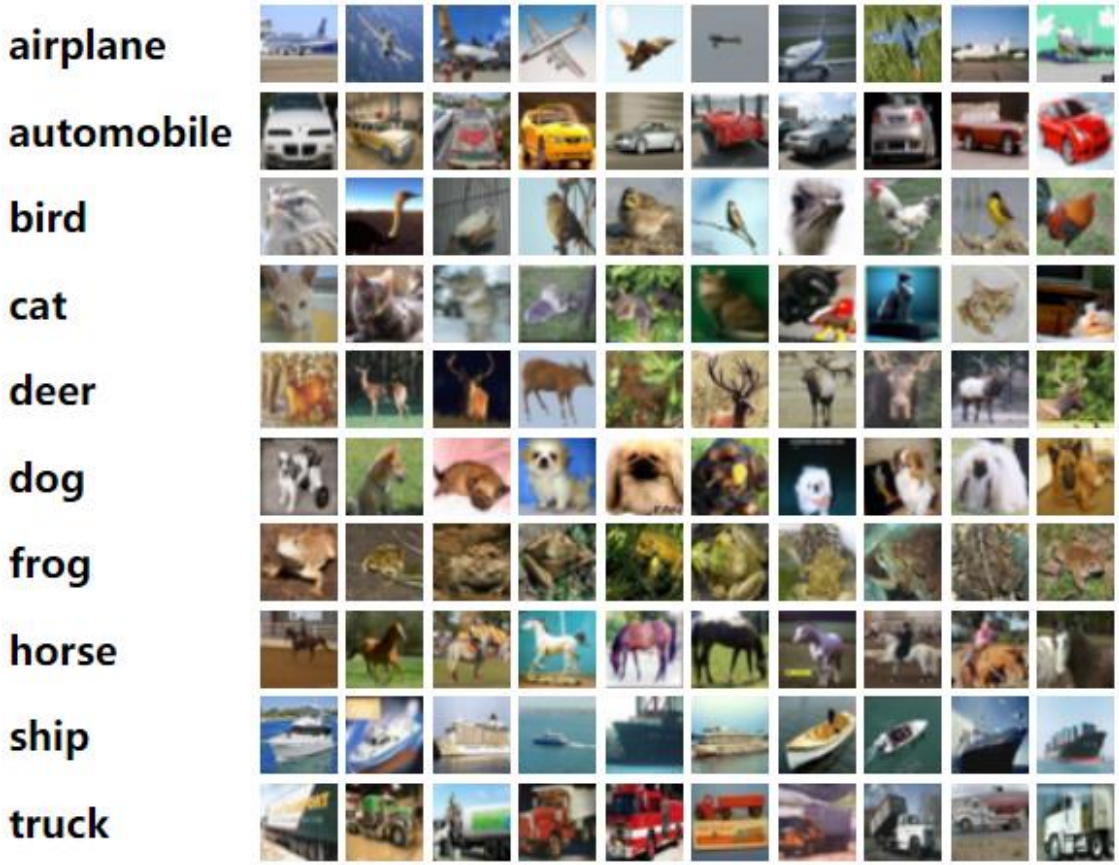
Convolutional and pooling layers learn local representations and reduce dimensionality.

Efficiently with sparse connections and parameter sharing.

Flattening and fully connected linear layer with nonlinear activation (like an MLP) to learn a global representation as a function of the local.

Final linear decision/classificatio n layer – softmax if you need to normalize.

What Kind of Data Do We Need for Image Recognition?



- Image input X
- With target labels for their class y.

- Model predicts a class $\hat{y}$ given X
- Evaluate prediction on cross entropy loss over classes

Reminder: Training = Find Parameters to Minimize Loss/Error Function on Training Data

Regression

mean squared error

$$\mathrm{MSE} = \frac{1}{m}\sum_{j=1}^{m}\left(\hat{y}^{(j)} - y^{(j)}\right)^2$$

Where $y^{(j)}$ is the target and $\hat{y}^{(j)}$ is the model prediction for the j'th data point out of m total.

Classification

mean cross entropy

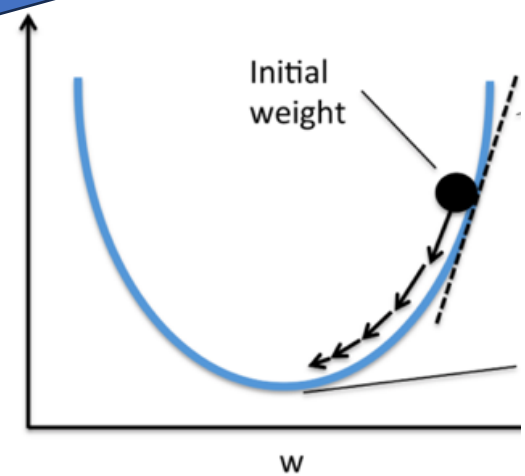
$$-\frac{1}{m}\sum_{j=1}^{m}\left(\sum_{i=1}^{k} y_i^{(j)} \ln\left(\hat{p}_i^{(j)}\right)\right)$$

Where $y_i^{(j)} = 1$ if the target class is i and 0 otherwise, and $\hat{p}_i^{(j)}$ is the model's estimate of $\Pr\left(y_i^{(j)} = 1 \mid x^{(j)}\right)$.

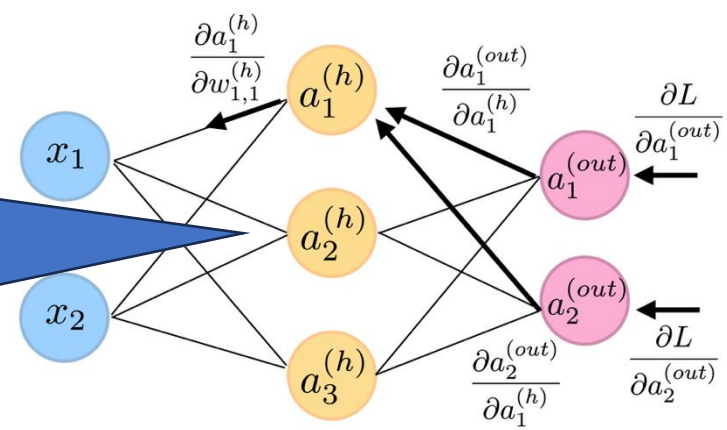
Reminder: How to Train = Learn Model Parameters?

Same algorithm as for the multilayer perceptron!

- The short answer is…
  - Minibatch Stochastic Gradient Descent to minimize the loss on training data
  - With Backpropagation to Calculate Derivatives of the loss with respect to model parameters



Same algorithm but backpropagation equations will be more complicated for a deeper architecture



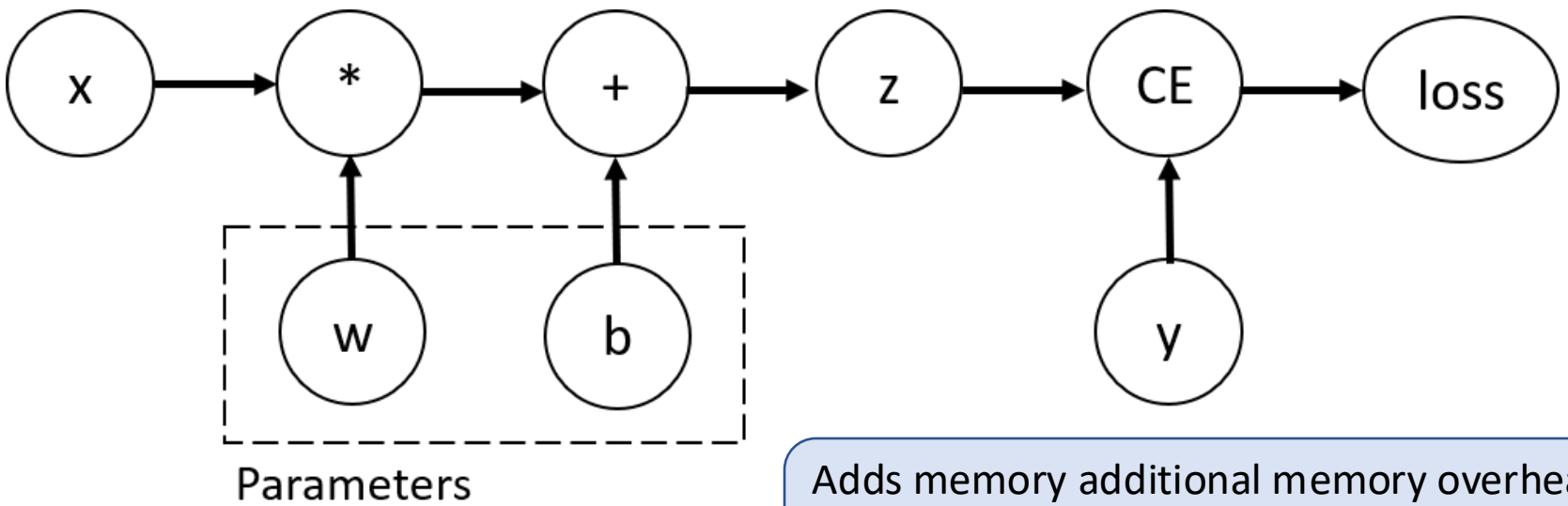
Gradient for hidden layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} + \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}}$$

Reverse Mode Automatic Differentiation in PyTorch

```
import torch
x = torch.randn(5) # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

For complex networks? Automatic differentiation performs backprop on the symbolic computational graph.



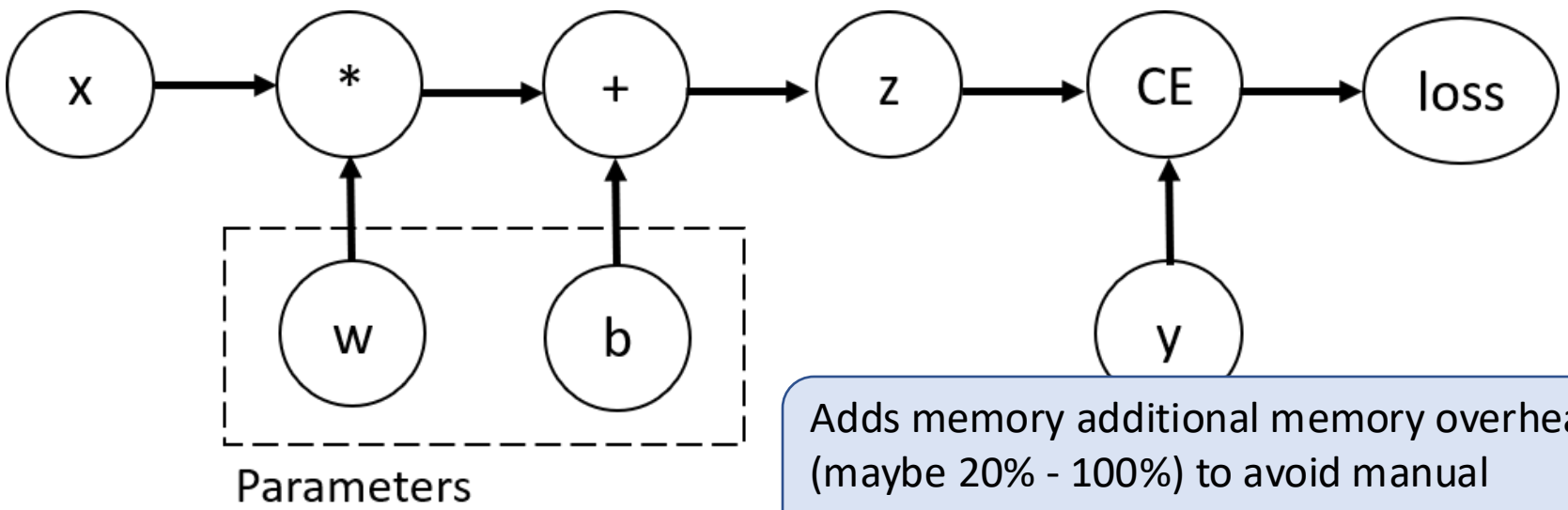
Adds memory additional memory overhead (maybe 20% - 100%) to avoid manual backprop derivations. Ubiquitous in practice.

Automatic Differentiation in PyTorch

```
loss.backward() # computes derivatives
print(w.grad)   # print derivatives for w params
print(b.grad)   # print bias term derivatives
```

```
tensor([[ 0.1246, 0.1249, 0.0962],
        [-0.2544, -0.2550, -0.1963],
        [ 0.5300, 0.5311, 0.4089],
        [-0.3047, -0.3053, -0.2351],
        [-0.4536, -0.4546, -0.3500]])
tensor([0.3312, 0.3319, 0.2556])
```

For complex networks? Automatic differentiation performs backprop on the symbolic computational graph.



Adds memory additional memory overhead (maybe 20% - 100%) to avoid manual backprop derivations. Ubiquitous in practice.

Stopping Gradient Tracking in PyTorch

Update tensor:

```
z.requires_grad = False
```

Or simply disable gradient computation:

```
with torch.no_grad():
    z = torch.matmul(x, w)+b

loss.backward() # computes derivatives
```

0.0s

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[11], line 4
      1 with torch.no_grad():
      2     z = torch.matmul(x, w)+b
----> 4 loss.backward()
```

Will get an error if calling backward without having done a gradient tracking forward pass to compute intermediate variables!

Training with PyTorch: Datasets

A PyTorch Dataset is an *object* that stores a collection of data, usually as a list of (input tensor, label) tuples and can be indexed like a list.

```
from torchvision import datasets, transforms
train_data = datasets.MNIST(root="data",
                            train=True,
                            download=True,
                            transform=transforms.ToTensor())
```

```
print(len(train_data))
input, label = train_data[0][0], train_data[0][1]
print(input.shape)
print(label)
```

60000

torch.Size([1, 28, 28])

5

Training with PyTorch: Dataloaders and Splits

A PyTorch DataLoader is an *iterable* over *batches* from a Dataset which can optionally shuffle data after each epoch.

```
from torch.utils.data import DataLoader
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
for X_batch, y_batch in train_loader:
    print(X_batch.shape)
    print(y_batch.shape)
    break # just previewing one example
```

torch.Size([64, 1, 28, 28]) torch.Size([64])

May want a separate train and validation set.

```
from torch.utils.data import random_split
train_data_sub, val_data = random_split(train_data, lengths=[0.8, 0.2])
train_dl = DataLoader(train_data_sub)
```

Example PyTorch Neural Network: torch.nn module

```python
from torch import nn

class ANN(nn.Module):
    def __init__(self):
        super().__init__()
        self.h = nn.Linear(in_features=20,
                           out_features=100,
                           bias=True)
        self.o = nn.Linear(in_features=100,
                           out_features=5,
                           bias=True)
```

Object-oriented module for defining neural networks

Inherits from nn.Module, base class for all PyTorch neural networks

Parent class constructor call

Object-Oriented API, convenient layer definitions

Example Forward Propagation with torch.nn

```
    def forward(self, x):
        a_h = self.h(x)
        z_h = nn.functional.relu(a_h)
        a_out = self.o(z_h)
        return a_out

    def predict(self, x):
        logits = self.forward(x)
        probs = torch.softmax(logits, dim=1)
        predicted = torch.argmax(probs, dim=1)
        return probs, predicted

X = torch.rand(size=(64, 20))
model = ANN()
probs, predicted = model.predict(X)
```

Alternative *functional* API for layers/activations

Unnormalized outputs

dim=1 → get probs for each input data point.

A complete PyTorch Neural Network specification **MUST IMPLEMENT** __init__ and forward (other methods optional).

Training with Pytorch: A Minimal Implementation

```python
model = ANN()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
epochs = 5

for epoch in range(epochs):
    for (inputs, labels) in train_dl:
        # Forward propagation
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)

        # Backward Propagation
        loss.backward()

        # Gradient step
        optimizer.step()
        optimizer.zero_grad()
```

Momentum optional

Must explicitly zero out previous gradient

Tracking Progress: Training Loss

```python
batches_per_eval = 200

for epoch in range(epochs):
    for batch, (inputs, labels) in enumerate(train_dl):
        # Forward propagation
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)

        # Backward Propagation
        loss.backward()

        # Gradient step
        optimizer.step()
        optimizer.zero_grad()

        if batch % batches_per_eval == 0:
            print(f'epoch {epoch}, batch {batch:>4}, loss: {loss.item():>3f}')
```

loss.item() to convert single element tensor to normal float type

Tracking Progress: Validation Loss and Accuracy per Epoch

```
24      with torch.no_grad():
25          val_loss = 0
26          correct_predictions = 0
27          for (inputs, labels) in val_dl:
28              outputs = model(inputs)
29              val_loss += loss_fn(outputs, labels).item()
30              correct_predictions += (outputs.argmax(1) == labels).sum().item()
31
32          accuracy = correct_predictions / len(val_dl.dataset)
33          print(f'epoch number: {epoch}')
34          print(f'validation loss: {val_loss:>5f}')
35          print(f'validation accuracy: {accuracy:>5f}\n')
```

Accuracy as fraction of correct predictions: Those where the index maximizing the probability estimate equals the true label.

epoch number: 0
validation loss: 138.461299
validation accuracy: 0.832250

epoch number: 1
validation loss: 89.635791
validation accuracy: 0.879833

...

Saving and Reloading a Trained Model / Model Parameters

Keep in mind that **the Jupyter Python kernel will *not* save your trained model weights between sessions.**

```
torch.save(model.state_dict(), 'model.params')
```

Then reload by creating a model of the same class and loading the parameters back in from the file.

```
model = ANN()
model.load_state_dict(torch.load('model.params’,
                              weights_only=True))
```

Additional Standard CNN Layers with torch.nn

```
self.conv1 = nn.Conv2d(in_channels=1,
                       out_channels=5,
                       kernel_size=(5, 5),
                       stride=1,
                       padding='same')
```

Grayscale image, would be 3 for color

Pads to maintain original image size

Optional, if omitted the output will have different dimensions than the input

```
self.pool = nn.MaxPool2d(kernel_size=(2, 2),
                         stride=2)
```

The most common max pooling: Reduces size by 2x in both dimensions

```
self.flatten = nn.Flatten()
```

Before fully connected layers

Parallelization and Matrix Multiplication

Specifications	Intel® Core™ i9-11900KB Processor	NVIDIA GeForce® RTX™ 3080 Ti
Base Clock Frequency	3.3 GHz	1.37 GHz
Cores	16 (32 threads)	10240
Memory Bandwidth	45.8 GB/s	912.1 GB/s
Floating-Point Calculations	742 GFLOPS	34.10 TFLOPS
Cost	~ $540.00	~ $1200.00

Three important observations:

1. Most computationally intensive operations during training are matrix/vector multiplications/additions.
2. These operations are *extremely* parallelizable.
3. Graphics processing units (GPUs) are highly parallel computer processors.

GPUs are overwhelmingly the preferred hardware for use with deep learning.

Speedup?

```python
import torch
from time import time

n = 1000
trials = 5
elapsed = 0.0

mps_device = torch.device("mps")
for t in range(trials):
    t1 = torch.rand((n, n), device = mps_device)
    t2 = torch.rand((n, n), device = mps_device)

    before = time()
    t3 = t1 @ t2
    after = time()
    elapsed += (after - before)

print(f'Took {(1000*elapsed)/trials:.3f} ms on GPU.')

for t in range(trials):
    t1 = torch.rand((n, n))
    t2 = torch.rand((n, n))

    before = time()
    t3 = t1 @ t2
    after = time()
    elapsed += (after - before)

print(f'Took {(1000*elapsed)/trials:.3f} ms on CPU.')
```

Just multiplying two 1,000 by 1,000 matrices in PyTorch; the task you expect maximum possible speedup from parallelization.

Took 0.147 ms on GPU.
Took 16.062 ms on CPU.

> 100x speedup (on a weak Apple m1 GPU)

Mileage may vary: Speedups between 2x and 100x are likely.

Devices, CUDA, and PyTorch

```python
T = torch.tensor([[1, 2],
                  [3, 4]])
T.device
```

✓ 0.0s

```
device(type='cpu')
```

By default, tensors are ready for computation on your CPU

```python
gpu = None

if torch.cuda.is_available():
    gpu = torch.device("cuda")
elif torch.backends.mps.is_available():
    gpu = torch.device("mps")

print(gpu)
```

✓ 0.0s

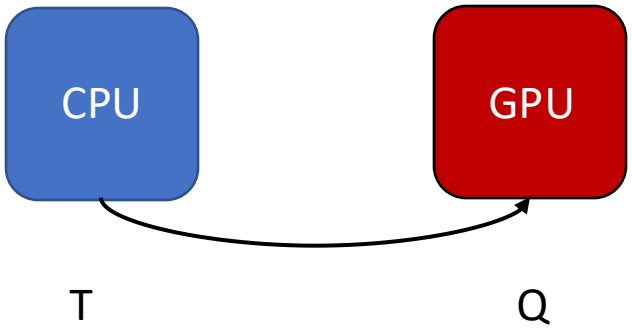
```
mps
```

Check if you have GPU resources. For example: cuda for NVIDIA devices, mps for Apple Silicon.

Moving Tensors & Computation

```
1  print(T.device)
2  Q = T.to(gpu)
3  print(Q.device)
```

✓ 0.0s




```
cpu
mps:0
```

```
1  T + Q
```

⊗ 0.0s

Can only compute with all tensors on the same device

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent
Cell In[47], line 1
----> 1 T + Q

RuntimeError: Expected all tensors to be on the same device, but
```

Efficient GPU Training

```python
model = CNN().to(gpu)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
epochs = 10

if gpu != None:
    for epoch in range(epochs):
        for batch, (inputs, labels) in enumerate(train_dl):
            inputs = inputs.to(gpu)
            labels = labels.to(gpu)

            # Forward propagation
            outputs = model(inputs)
            loss = loss_fn(outputs, labels)

            # Backward Propagation
            loss.backward()

            # Gradient step
            optimizer.step()
            optimizer.zero_grad()
```

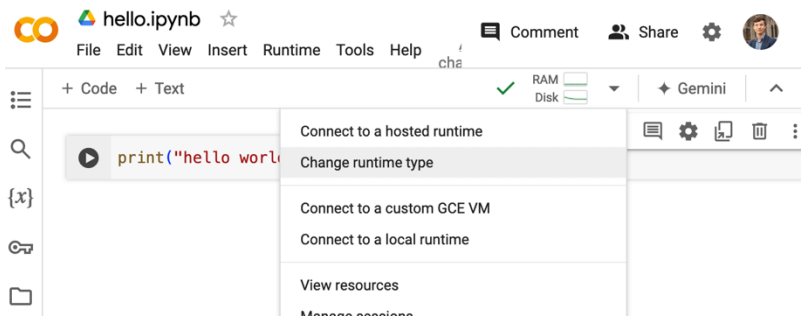
Transfer model to the GPU

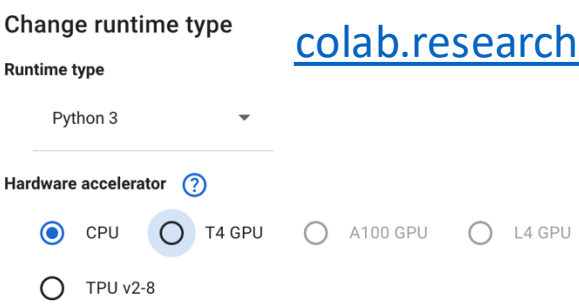
And each batch of data. (GPU usually memory limited!)

No other changes, enjoy the speedup!!!

GPU Resources

Free and easy to use cloud jupyter notebooks in Google Colab with cuda. Limited in runtime.





colab.research.google.com

May have local GPU (Apple Silicon with mps, etc.)

OR: Computer Science Department Compute Cluster

Duke TRINITY COLLEGE OF ARTS AND SCIENCES | DEPARTMENT of COMPUTER SCIENCE

Cluster Computers

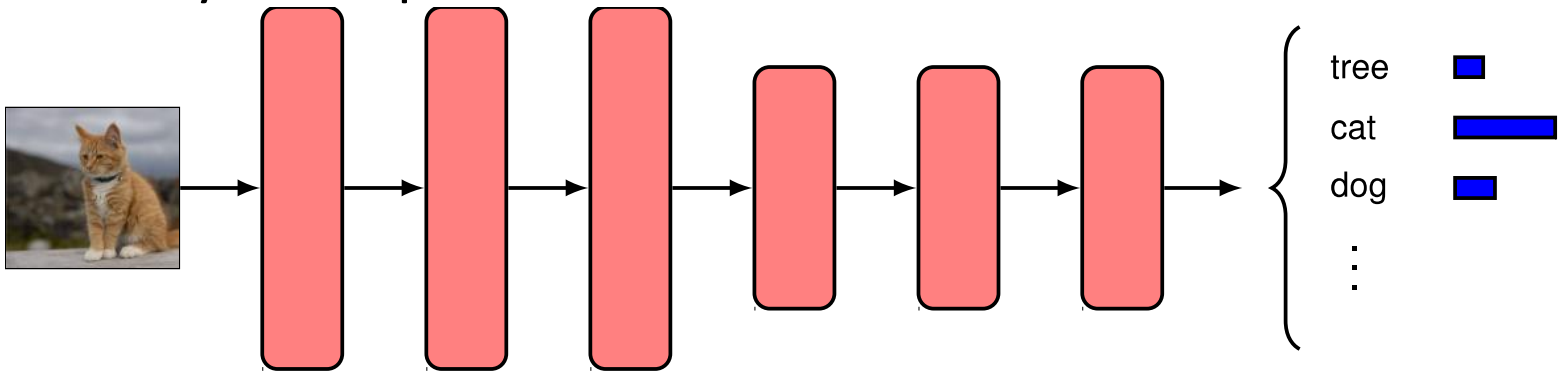
The C. S. Department maintains a small cluster of 91 (2336 cores) machines for general computing. The machines are set up to use Slurm for batch job submission. **Be aware**

GPU	Cores	Tensor Cores	VRAM	Hosts
16 A6000s	10752	336	48GB	compsci-cluster-fitz
24 a5000s	8192	256	24GB	linux[41-44]
10 V100s	5120	640	32GB	gpu-compute[5-7]
26 P100s	3584		12GB	linux[41-50]
				gpu-compute[4-5]
24 K80s	4992		12GB	gpu-compute[1-3]
30 2080RTXTi	4352		11GB	linux[41-60]

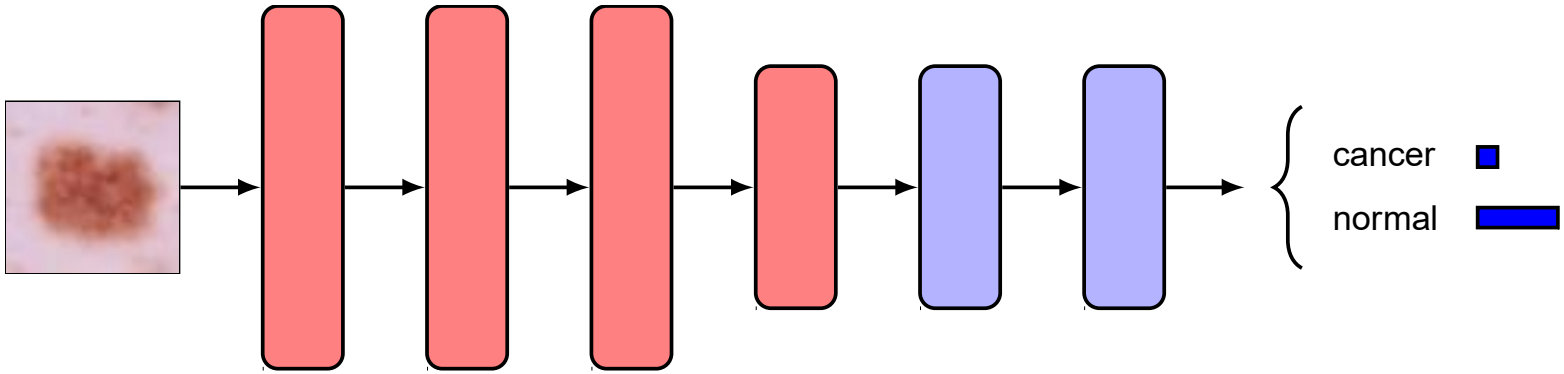
cs.duke.edu/csl/facilities/cluster

Transfer Learning for Deep Network Architectures.

The features representations used in hidden layers for one task may be helpful for a different but related task.

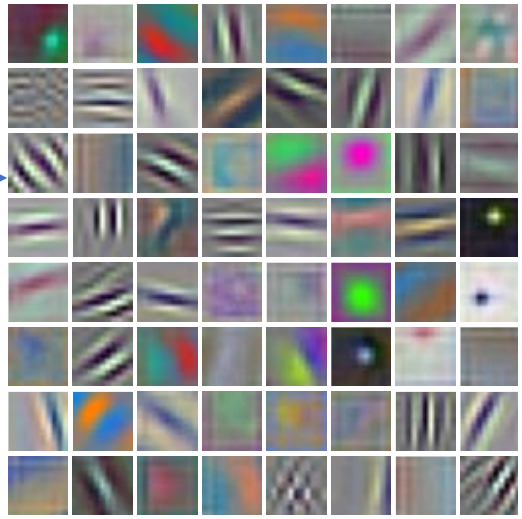


In which case we can take the pre-trained model for one task then *replace and retrain its decision layer* for a different task (**feature extraction**).



Why Does This Work?

Visualization of various learned convolutional filters in first layer.



Validation images with highest activations by filter at different layers. Increasing abstraction.

Layer 1

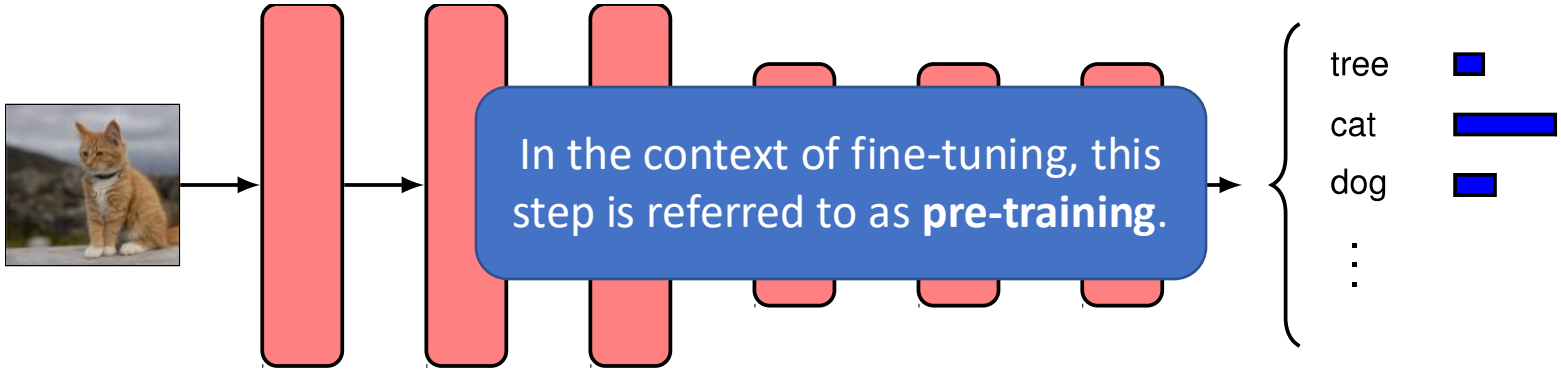
Layer 2

Layer 3

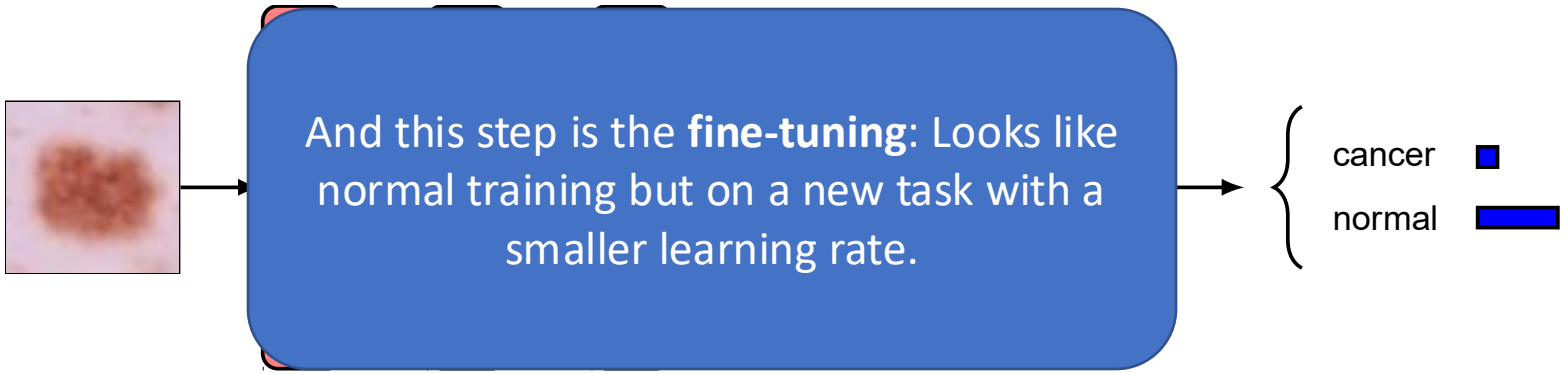
Layer 5

Fine-Tuning

Or we can view the pretrained weights of the original model as *a* ***good initialization*** for a related task.



And retrain ALL model parameters on a new task, using a (relatively) small learning rate (**fine-tuning**).



Inspecting a PyTorch Model

```python
model = CNN()
print(model)
```

✓ 0.0s

```
CNN(
  (conv1): Conv2d(1, 5, kernel_size=(5, 5), stride=(1, 1), padding=same)
  (conv2): Conv2d(5, 15, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (pool): MaxPool2d(kernel_size=(2, 2), stride=2, padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (h): Linear(in_features=735, out_features=100, bias=True)
  (o): Linear(in_features=100, out_features=10, bias=True)
)
```

```python
for param_tensor in model.parameters():
    print(param_tensor.size())
```

✓ 0.0s

```
torch.Size([5, 1, 5, 5])
torch.Size([5])
torch.Size([15, 5, 3, 3])
torch.Size([15])
torch.Size([100, 735])
torch.Size([100])
torch.Size([10, 100])
torch.Size([10])
```

$125 = 5 \times 1 \times 5 \times 5$ trainable parameters (coefficients, 1st conv layer)

5 trainable parameters (biases in first conv layer)

73,500 trainable parameters (coefficients, 1st fully connected layer)

Modifying a PyTorch Model Architecture (e.g., for Fine-Tuning)

```python
print(model)
```

✓ 0.0s

```
CNN(
  (conv1): Conv2d(1, 5, kernel_size=(5, 5), stride=(1, 1), padding=same)
  (conv2): Conv2d(5, 15, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (pool): MaxPool2d(kernel_size=(2, 2), stride=2, padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (h): Linear(in_features=735, out_features=100, bias=True)
  (o): Linear(in_features=100, out_features=10, bias=True)
)
```

```python
model.o = nn.Linear(100, 20)
print(model)
```

✓ 0.0s

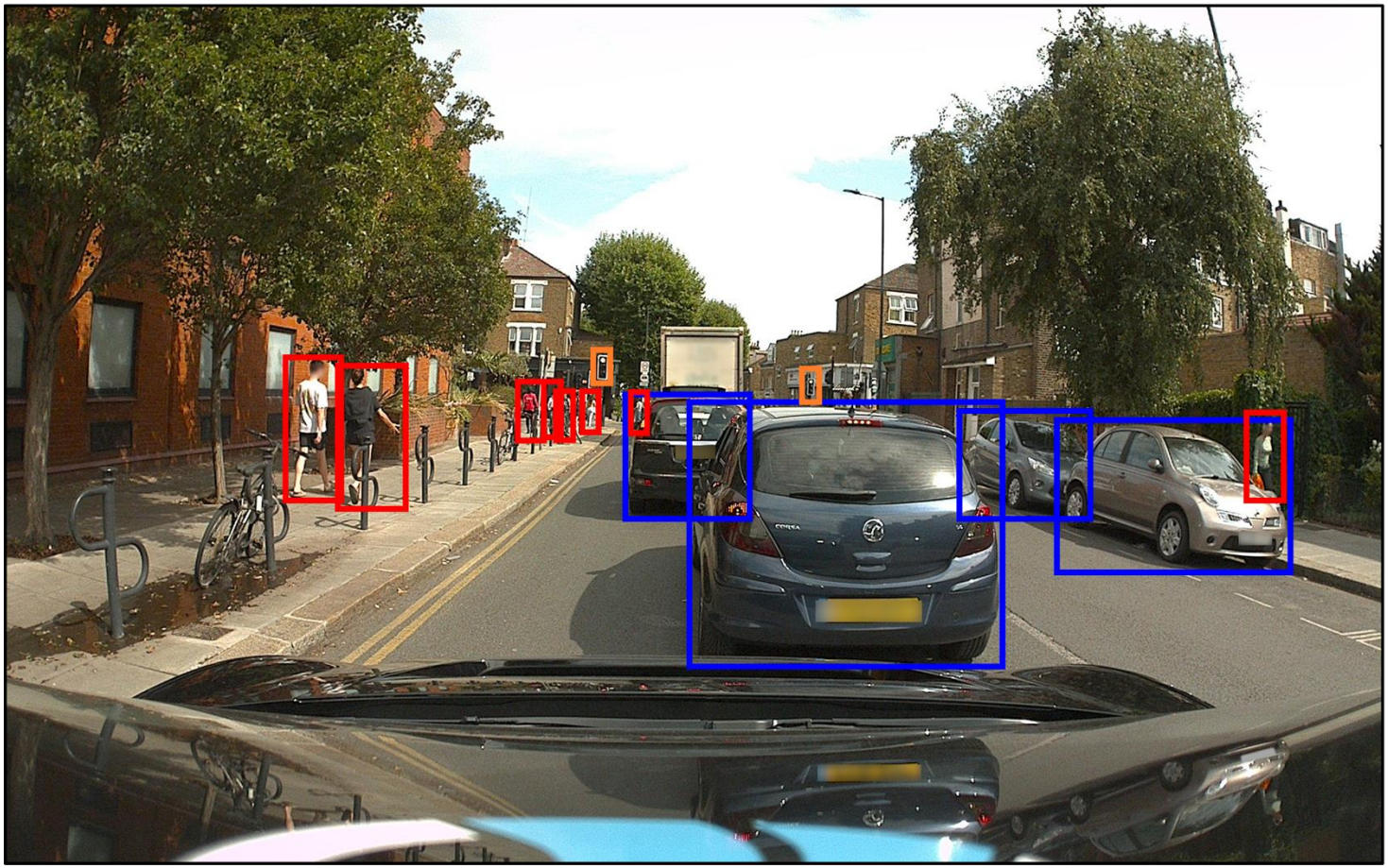
```
CNN(
  (conv1): Conv2d(1, 5, kernel_size=(5, 5), stride=(1, 1), padding=same)
  (conv2): Conv2d(5, 15, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (pool): MaxPool2d(kernel_size=(2, 2), stride=2, padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (h): Linear(in_features=735, out_features=100, bias=True)
  (o): Linear(in_features=100, out_features=20, bias=True)
```

Changing the output layer for a different task

Part 2: Object Detection

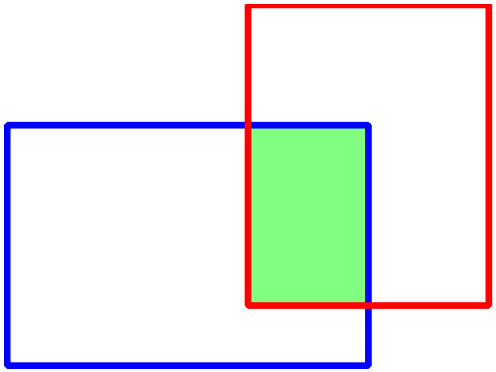
Object Detection as Usually Framed: Bounding Box Prediction

Predict a *bounding box* around an object of interest.

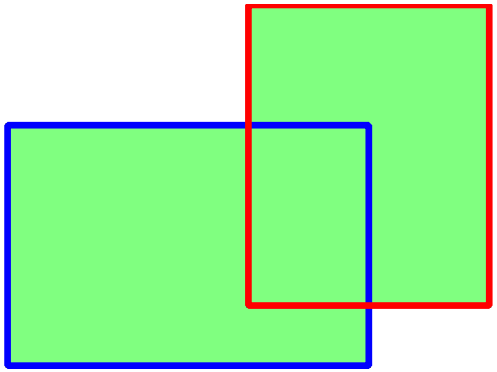


Evaluating Bounding Boxes: Intersection over Union Metric

Number of pixels in *both bounding boxes* (intersection) divided by number of pixels in *either bounding box* (union).



area of intersection



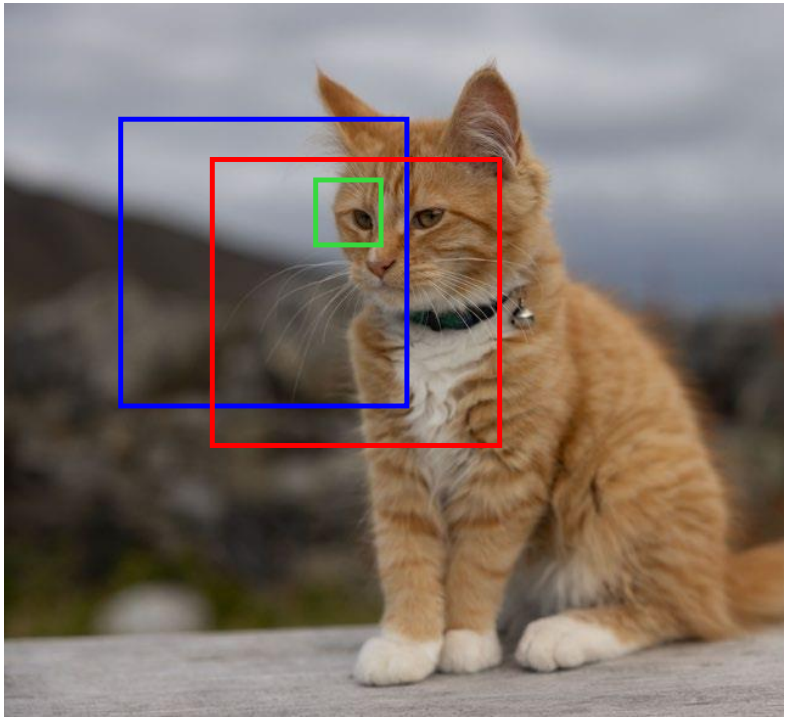
area of union

Typically consider a prediction "correct" if IoU > 0.5.

Labeled training data has human-drawn "correct" example bounding boxes.

Sliding Window Approaches with Convolutional Nets

Given an object recognition model, can “slide” a bounding box over the image, at each position predicting probability of the object in question.



Reduces bounding box prediction to image recognition.

Good localization but high runtime overhead (many forward propagations in object recognition CNN model).

More Sliding Window Challenges: Determining the Bounding Box Shape

1. Use multiple differently shaped sliding windows,
2. Or slide the same window over multiple differently resized copies of the image.

(a) (b) (c)

Adding more computational overhead (~10,000 forward passes per input).

Is there a way to predict bounding boxes with a single forward pass (trading off speed for precision?)

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the version available on IEEE Xplore.

52,000 citations

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†
University of Washington*, Allen Institute for AI†, Facebook AI Research¶
http://pjreddie.com/yolo/

Can see the full paper at arxiv.org/abs/1506.02640

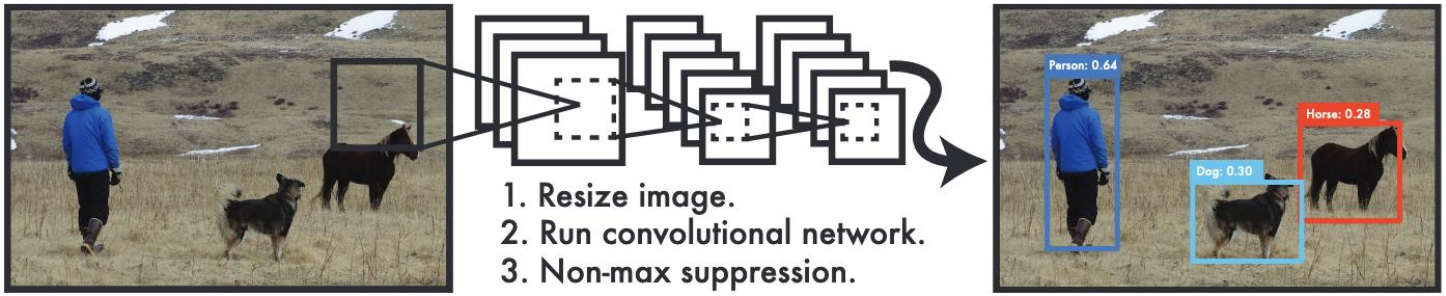


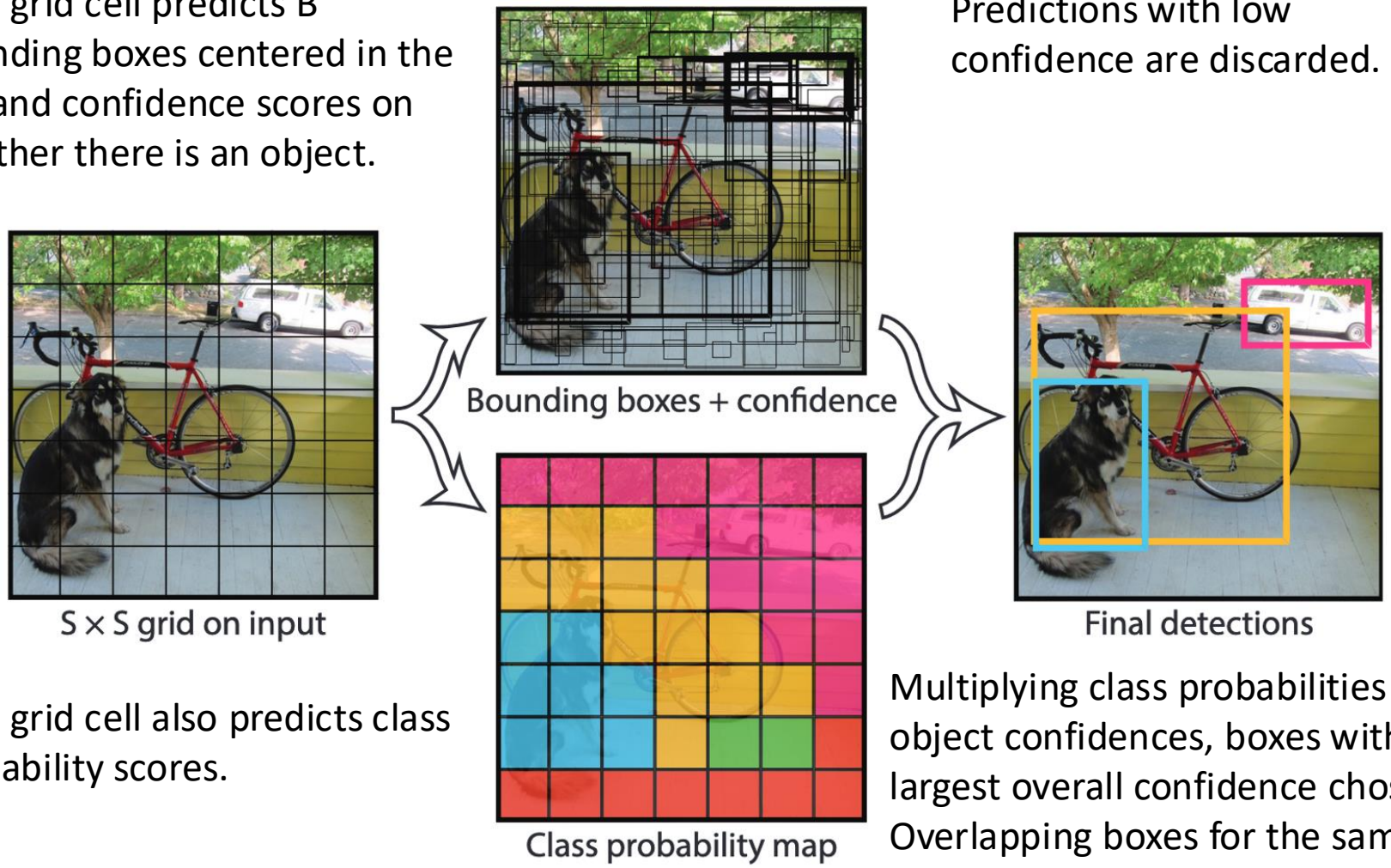
Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448 × 448, (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

Commonly applied for real-time detection systems, especially video recording.

YOLO Method

Each grid cell predicts B bounding boxes centered in the cell and confidence scores on whether there is an object.

Predictions with low confidence are discarded.



Each grid cell also predicts class probability scores.

Multiplying class probabilities and object confidences, boxes with largest overall confidence chosen. Overlapping boxes for the same class are discarded.

The YOLO Convolutional Neural Network Architecture

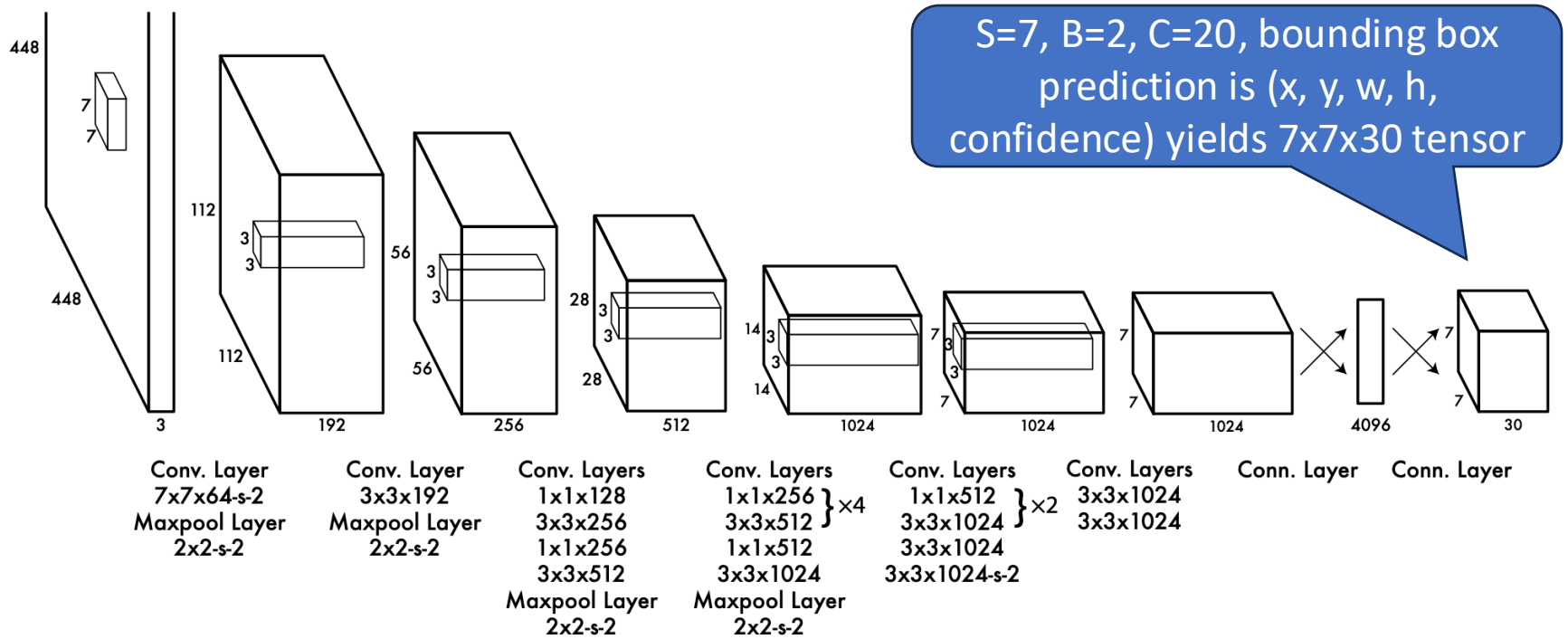


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

Real Time Object Detection with YOLO Demo

Image Segmentation

Predict a distribution over classes *for each pixel*.

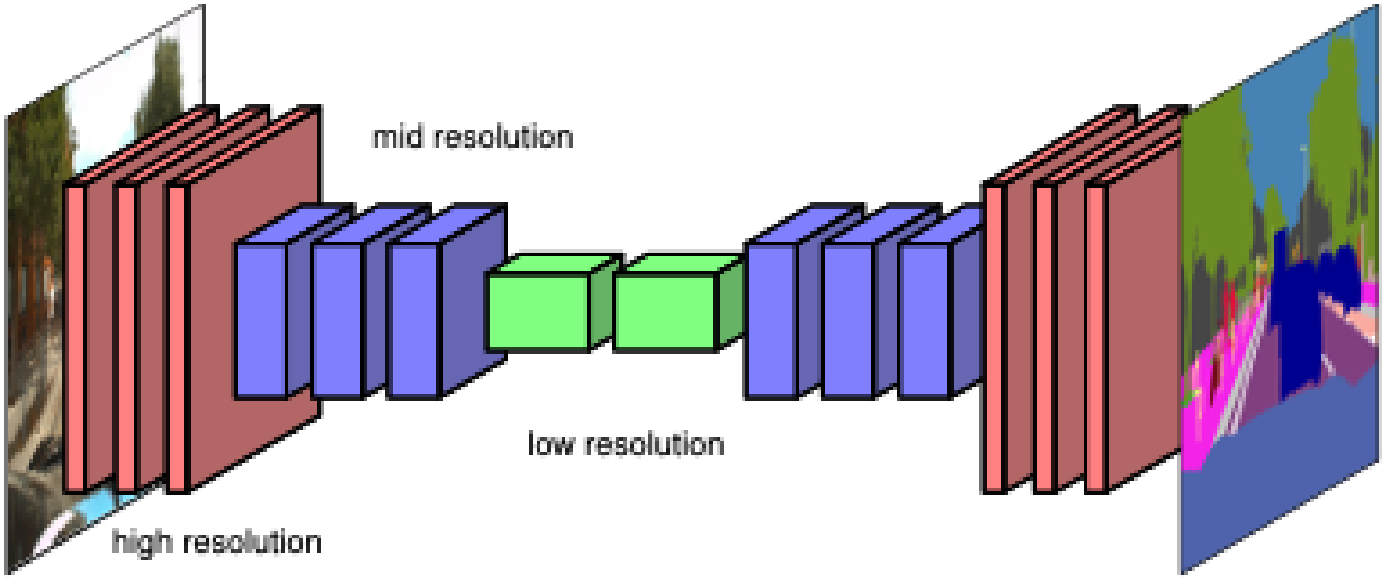
Visualize maximum probability class per pixel, color coded.

Model output tensor has *same resolution as input.*

Requires coded images by hand – labor intensive but more fine-grained results than bounding boxes (common in applications like medical imaging).

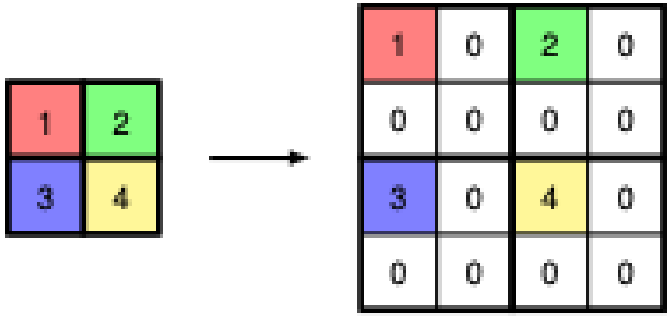
Convolutional Networks for Image Segmentation

After convolutional and pooling layers, need to up-sample back to original resolution.



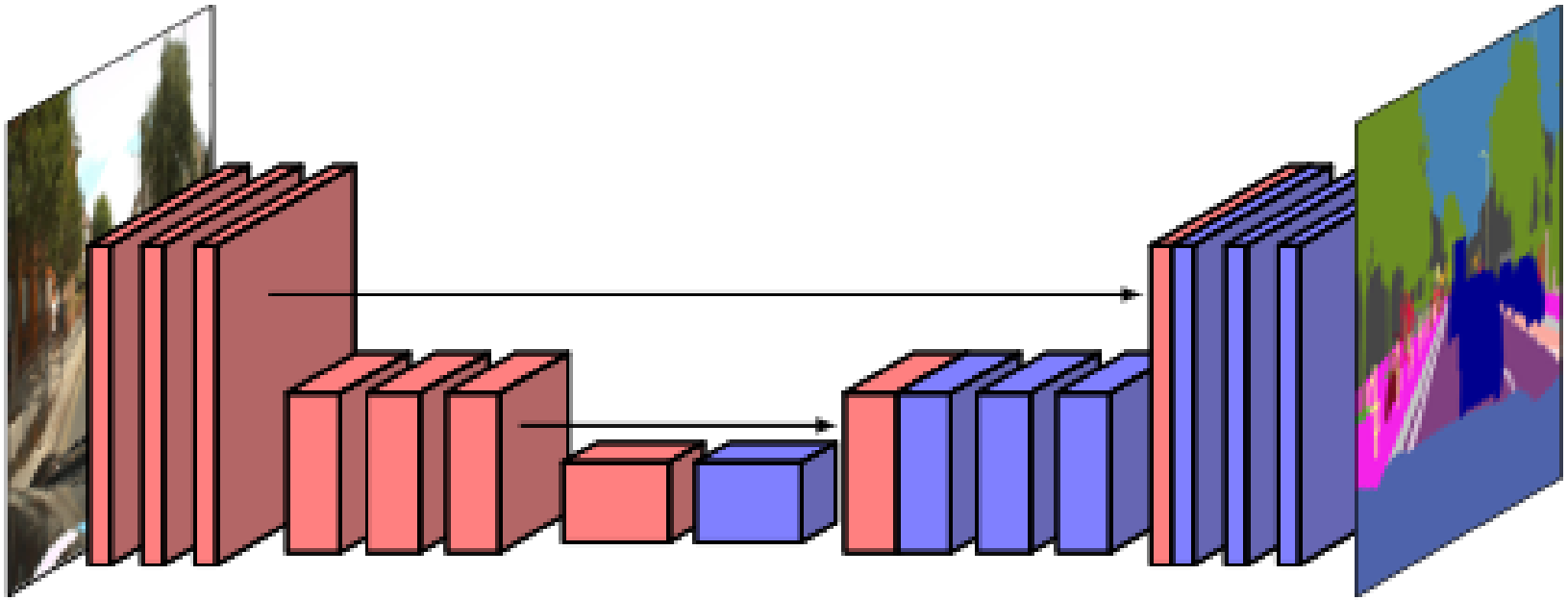
For example, max-unpooling:

(typically saves original pixel position from prior pooling step)



U-net Architecture

Designed to improve the spatial resolution of the up-sampling.



Pair each up-sampling layer with a down-sampling layer -- up-sampling layer takes two inputs.